

第二节&第三节

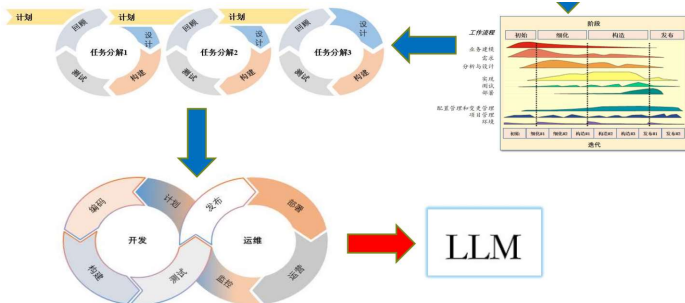
2025年6月5日 22:14

1. 软件模型的演化流程：边做边改→瀑布→V→原型→增量→螺旋→敏捷

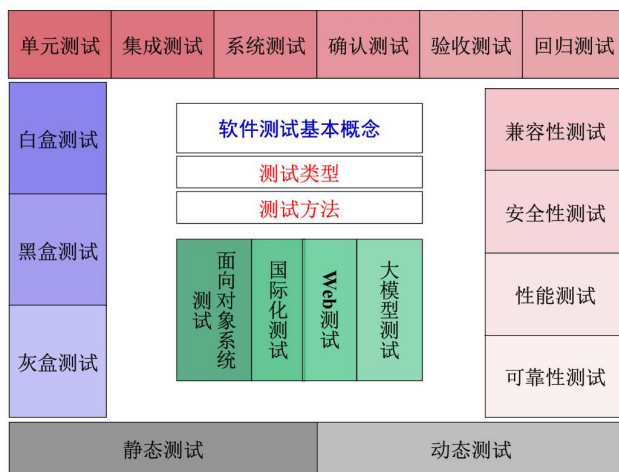
→UP→DevOps

宏观演变内容：
黑盒 → 灰盒 → 白盒
线性 → 非线性
惯例 → 敏捷

V模型→W模型↓

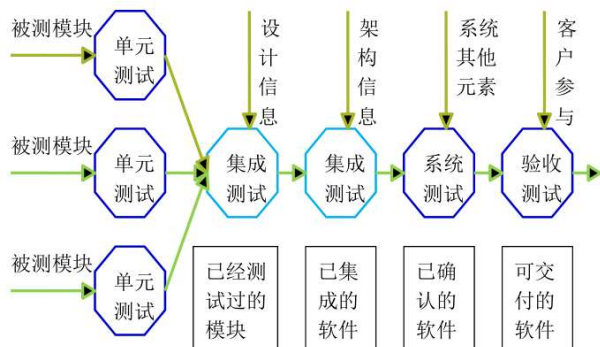


2. 软件测试分类：
- 静态、动态
 - 白盒、黑盒、灰盒
 - 单元、集成、系统、验收、回归
 - 功能、性能、安全、可靠性
- 测试过程：
需求描述
计划
用例生成（设计）
测试开发
执行
结果与预期结果比较
充分性评估
报告生成
- 测试方法：
蜕变测试
随机测试
组合测试
基于模型的测试
- 测试应用：
构建软件测试
OO测试
Web测试
服务测试
云测试
- ←面向测试任务



1. 开发者测试：开发者（有别于来自测评机构的专职测试人员）所做的测试。目标是在软件交付（验收测试）前发现和解决绝大多数代码缺陷。
理论依据：“前段发现问题的代价远小于后端”

测试阶段的信息流：除了测试本身的信息，开发阶段的一些内容也是测试输入：



3. 集成测试：验证多个模块、组件或服务在协同工作时的交互是否符合预期。关注单元间接口、数据传递、依赖关系和整体功能逻辑，确保组合后的系统部分能正确运行。

原因：
一个模块会对另一个模块产生不利影响
子功能合成不一定产生预期主功能
独立可接受误差组装后会超过可接受限度
可能发现单元测试中未发现的接口错误
单元测试无法发现时序/资源竞争问题

核心概念：

2. 单元测试：验证代码中最小可独立测试单元的行为是否符合预期（黑/白盒）

核心概念：

- （1）测试对象：代码中的“单元”常指函数、方法或类
- （2）独立性：测试时隔离外部依赖，使用模拟或桩技术，确保测试仅关注当前单元的逻辑
- （3）自动化：通过测试框架自动执行，便于集成到开发流程中

实施步骤：

- （1）编写测试用例：覆盖正常/异常输入、边界条件
- （2）运行测试：使用框架命令进行测试
- （3）验证结果：断言预期结果与实际结果是否一致

与其他测试的区别：

集成测试	验证多个模块协作，涉及真实依赖
系统测试	评估整个系统是否符合需求

FIRST原则：
Fast 快速
Isolated 独立
Repeatable 可重复
Self-Validating 自验证
Timely 及时

可利用工具统计代码覆盖率，但避免盲目追求高覆盖率，注重关

键逻辑覆盖。单元测试内容：
单元接口
数据结构
边界条件
独立执行
错误处理

核心概念：

(1) 测试对象：模块间接口、子程序或微服务间交互、第三方服务集成

(2) 测试范围：单元测试<集成测试<系统测试。小规模（2模块）/中规模（多组件）

(3) 依赖管理：部分依赖使用真实服务，部分使用模拟工具。

核心目的： $\left\{ \begin{array}{l} \text{暴露接口问题} \\ \text{验证协作逻辑} \\ \text{支持持续集成} \end{array} \right.$

集成测试类型：

(1) 非增式集成（大爆炸集成）：对所有模块进行个别的单元测试后，按程序结构图将各模块联接起来，把联接后的程序当做一个整体进行测试

(2) 增式集成：把下一个待测模块同已测试号的模块结合起来进行测试，一次增加一个待测模块。

桩模块：软件测试中用于模拟被测模块所调用的其他模块的虚拟模块。主要作用是代替被测模块的实际依赖模块，以解决被测模块在集成测试中因依赖模块未完成或未测试而无法进行测试的问题
自顶向下增式集成：

(1) 主控模块作为测试驱动，所有与主控模块直接相连的模块作为桩模块

(2) 根据集成的方式（深度/广度），每次用一个替换从属的桩模块

(3) 在每个模块被集成时，都必须已经进行了单元测试

(4) 进行回归测试以确定集成新模块后没有引入错误，重复第二步直到整个系统集成完毕

驱动模块：软件测试中用来模拟被测模块的上一级模块，相当于被测模块的主程序。主要作用是接收数据，将相关数据传送给被测模块，启动被测模块，并打印出相应的结果。驱动模块的主要目的是访问类库的属性和方法，检测类库的功能是否正确。

自底向上增式集成：

(1) 从最底层模块开始，组合成一个构件，用以完成指定的软件子功能

(2) 编制驱动模块，协调测试用例的输入/输出

(3) 测试继承后的构件

(4) 按程序结构向上集成测试后的构件，同时除去驱动模块

对比：

	非增式	增式
工作量	大	小
接口错误发现时间	晚	早
错误定位	难	易
测试程度	不彻底	彻底
测试并行性	好	差

	优点	缺点
自顶向下	可以做到逐步求精，可让测试者看到系统框架	需要桩模块；输入/输出模块接入前桩模块表示测试数据有一定困难；桩模块不能模拟数据，若模块间数据流不能构成有向无环图，一些模块的测试数据难以生成；观察和解释测试输出困难。
自底向上	驱动模块模拟了所有调用参数，即使数据流并未构成有	直到最后一个模块被加进去之后才能看到整个程序（系统）的框架；只有到测试过程后期才能发现时序和资源

4. 系统测试：验证完整的软件系统是否符合SRS中的功能/非功能需求。关注整个系统作为统一体的行为，而非单个模块/组件，在集成测试后，用户验收测试前执行。

应用场景： $\left\{ \begin{array}{l} \text{所有集成测试完成} \\ \text{软件系统间联合测试} \\ \text{软硬件联合测试} \\ \text{模拟真实运行环境的测试} \end{array} \right.$

端到端测试：从头到尾测试整个软件产品的方法，确保应用程序流程按预期运行。模拟真实用户场景，验证被测系统及其组件的集成和数据完整性，从最终用户的体验进行测试。

核心概念：

(1) 测试对象：完整的、已集成的软件系统（包括所有模块、服务和依赖项），部署在真实或者接近真实的环境中

(2) 测试范围：功能性/非功能性需求

(3) 测试视角：黑盒测试，端到端测试

核心目的： $\left\{ \begin{array}{l} \text{验证系统完整性} \\ \text{评估非功能指标} \\ \text{为验收测试铺路} \end{array} \right.$

实施步骤：

(1) 测试需求分析

(2) 设计测试用例

(3) 准备测试环境

(4) 执行测试

(5) 缺陷管理与回归

5. 验收测试：软件测试的最后阶段，验证系统是否满足用户需求、业务目标和合同条款，决定软件是否可交付给客户或上线使用。从最终用户或业务方的视角出发，确保软件真正解决了实际问题，而非仅通过技术验证

特征： $\left\{ \begin{array}{l} \text{V模型中测试的最后一道工序} \\ \text{用户在场或者直接测试} \\ \text{用户可能自定义测试用例} \end{array} \right.$

6. Alpha 测试：通常由用户或其他人（非开发/测试人员）完成

(1) 是在开发即将完成时对应用进行的测试，仍然允许对设计进行微小的变动

(2) 是由用户在开发环境下进行的测试，也可是开发机构内部的用户在模拟实际操作环境下进行的测试。开发者坐在用户旁边，是开发者受控的环境下进行的测试。由开发者随时记录下错误情况和使用中的问题。

Beta 测试：

(1) 一种验收测试，是一种软件发布前的测试。验收测试根据SRS严格检查产品，对照说明书对软件产品的各方面要求，确保所开发的软件产品符合用户的各项要求

(2) 是软件的多个用户在一个或多个用户的实际使用环境下进行的测试。开发者通常不在现场，不能由程序员或测试员完成

7. 回归测试：确保代码修改不会破坏现有功能的测试方法。目的是验证在添加新功能、修复缺陷或优化代码后，系统已有的功能仍能正常工作。

原因：软件被修改时候，新的代码可能无意中引入错误，影响之前正常的功能。

目标： $\left\{ \begin{array}{l} \text{修改或增加的部分是正确的} \\ \text{没有引起其他部分产生错误} \end{array} \right.$

应用： $\left\{ \begin{array}{l} \text{增量开发} \\ \text{版本控制} \\ \text{软件维护} \end{array} \right. \text{测试方法：} \left\{ \begin{array}{l} \text{全部重测} \\ \text{按风险测试} \\ \text{按频率测试} \\ \text{按修改测试} \\ \text{按依赖测试} \end{array} \right.$

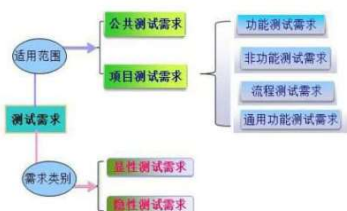
基本流程：

自底向上	驱动模块模拟了所有调用参数，即使数据流并未构成有向无环图，生成测试数据困难小；特别适合关键模块在结构图底部的情形	直到最后一个模块被加进去之后才能看到整个程序（系统）的框架；只有到测试过程后期才能发现时序和资源竞争问题
------	--	--

8. 测试需求：测试活动的基础，定义“测什么”。是从软件需求（功能/非功能）中提炼出来的、可测试的、明确的验证要求。来源于用户需求、系统需求和设计文档；是编写测试用例的依据；明确了测试范围和测试目标。

测试计划基本要素： $\left\{ \begin{array}{l} \text{测试需求 (What)} \\ \text{怎么测 (How)} \\ \text{测试时间 (When)} \\ \text{需要多少人 (Who)} \\ \text{测试环境 (Where)} \end{array} \right\} + \left\{ \begin{array}{l} \text{技能} \\ \text{工具} \\ \text{背景知识} \\ \text{可能遇到的风险} \end{array} \right\}$

表达方式：测试需求列表、测试需求跟踪矩阵



9. 测试计划：一份详细描述测试目标、范围、方法、资源、进度和交付物的文档，用于协调和组织测试活动。是测试过程的蓝图，由测试经理或测试负责人编写。

(i) 测试目标：

单元测试	验证程序最小单元有无错误、单元编码与设计是否吻合
集成测试	验证组成系统的模块接口有无错误、代码实现的设计与需求设计是否吻合
系统测试	检验组成整个系统的代码、以及系统的软硬件配合有无错误；代码实现的系统与用户需求是否吻合；检验系统的各种文档是否完整、有效；模拟验收测试的要求，检查系统是否符合用户的验收标准。
验收测试	使客户验收签字；系统是否符合事先约定的验收标准
回归测试	验证程序修改或者版本更新以后，以前正确的功能和其他指标仍旧正确。

(ii) 测试范围：功能测试、接口测试、性能测试、负载测试

(iii) 测试方法： $\left\{ \begin{array}{l} \text{代码覆盖策略} \left\{ \begin{array}{l} \text{决策覆盖} \\ \text{基于数据流的测试} \end{array} \right. \\ \text{基于规约的测试} \left\{ \begin{array}{l} \text{等价类划分} \\ \text{边界值分析} \\ \text{组合策略} \end{array} \right. \\ \text{基于状态的测试} \end{array} \right.$

(iv) 测试资源： $\left\{ \begin{array}{l} \text{角色} \left\{ \begin{array}{l} \text{角色名称} \\ \text{角色安排} \\ \text{责任} \end{array} \right. \\ \text{系统资源} \left\{ \begin{array}{l} \text{软件} \\ \text{硬件} \end{array} \right. \end{array} \right.$

(v) 交付物： $\left\{ \begin{array}{l} \text{测试计划} \\ \text{测试环境} \\ \text{测试包} \\ \text{测试日志} \\ \text{测试报告} \end{array} \right.$

(执行测试脚本

应用： $\left\{ \begin{array}{l} \text{版本控制} \\ \text{软件维护} \end{array} \right.$ 测试方法： $\left\{ \begin{array}{l} \text{按频率测试} \\ \text{按修改测试} \\ \text{按依赖测试} \end{array} \right.$

基本流程：

(1) 识别软件被修改部分

(2) 从原基线测试用例中排出不适用的测试用例，确定新的软件版本仍有效的测试用例，结果是建立一个新的基线用例测试库

(3) 根据一定策略从基线用例测试库中选择测试用例测试被修改的软件

(4) 如果必要，生成信的测试用例集，用于测试基线用例测试库无法充分测试的软件部分

(5) 用新的用例集执行修改后的软件

10. 测试设计： $\left\{ \begin{array}{l} \text{测试用例设计：确认和详细描述测试用例} \\ \text{测试脚本设计：确认和设计测试脚本} \\ \text{覆盖准则设计：评估测试覆盖} \end{array} \right.$

测试用例设计：

黑盒测试	使用单元接口和功能描述，不需了解被测单元的内部结构	基于规约的测试；等价类划分；边界值分析；状态转移测试
白盒测试	使用被测单元内部如何工作的信息	分支测试、条件测试、数据定义使用测试、内部边界值、测试
其他技术	不属于以上2类	错误猜测

测试脚本：一组自动化执行的指令或代码，用于模拟用户操作或验证软件功能。替代手动测试的重复步骤，通过编程语言或工具实现测试用例的自动执行，提高效率并减少人为错误。

核心作用： $\left\{ \begin{array}{l} \text{自动化重复操作} \\ \text{精准验证结果} \\ \text{支持持续集成} \end{array} \right.$

覆盖率指标： $\left\{ \begin{array}{l} \text{覆盖率} \\ \text{控制流覆盖率} \\ \text{数据流覆盖率} \\ \text{分支覆盖率} \\ \text{路径覆盖率} \\ \text{搭建测试环境} \\ \text{录制或编写测试脚本} \\ \text{开发测试驱动模块} \\ \text{开发桩模块} \\ \text{建立外部数据集} \end{array} \right.$

常用覆盖准则	
1 语句覆盖	保证程序中的每条语句都执行一遍
2 判定覆盖	保证每个判断取True和False至少一次
3 条件覆盖	保证每个判断中的每个条件的取值至少满足一次
4 判定条件覆盖	保证每个条件和由条件组成的判断的取值...
5 条件组合覆盖	保证每个条件的取值组合至少出现一次...
6 路径覆盖	覆盖程序中所有可能路径

11. 测试开发： $\left\{ \begin{array}{l} \text{搭建测试环境} \\ \text{录制或编写测试脚本} \\ \text{开发测试驱动模块} \\ \text{开发桩模块} \\ \text{建立外部数据集} \end{array} \right.$

搭建测试环境：目的是模拟真实运行环境，确保测试结果的准确性和可靠性。在此之前需要明确测试环境的目标。步骤：

(1) 确定测试类型

(2) 匹配需求：硬件、软件、数据、网络

建立测试环境的核心是模拟真实场景并保持可行性。通过容器化、自动化工具和严格的数据管理，可显著提高测试效率和可靠性，且测试环境与生产环境需要高度一致。

驱动程序和桩模块的作用：当被测模块依赖其他未完成或不可用的组件时，需通过驱动程序和桩模块模拟其行为，实现隔离测试。

13. 测试评估测试报告

评估方法： $\left\{ \begin{array}{l} \text{测试用例覆盖} \\ \text{代码覆盖} \\ \text{缺陷分析} \\ \text{图表} \end{array} \right.$

12. 测试执行和测试结果确认: { 执行测试脚本
测试执行情况分析
结果验证和确认

软件测试结果验证是确保测试结果准确、可靠的核心环节，其目标是确认测试结果真实反映软件质量，避免误判或遗漏关键问题。在软件测试中，比较测试结果和预期结果是验证软件功能是否正确核心步骤。

软件测试结果确认是确保测试活动有效性和软件质量的关键环节，目的是验证测试的完整性和准确性，并为发布决策提供可靠依据。

手动测试结果对比:

逐项检查法

适用: 业务流程简单、结果可直观观察的测试

步骤: { 执行测试用例, 记录实际结果
与用例文档中的预期结果逐项对比
标记差异点并分析原因

自动化测试结果对比

断言

原理: 在代码中预设条件, 判断实际结果是否符合预期

常用类型: 等于、包含、布尔

14. 静态测试（静态分析，对被测程序进行特性分析的方法总称）：不实际运行程序，通过检查和阅读来发现错误并评估代码质量。

类型:

- (1) 走查：开发组内部进行的，采用讲解、讨论和模拟运行的方式进行的查找错误的活动。
- (2) 审查：开发组内部进行的，采用讲解、提问并使用Checklist方式进行的查错活动。一般有正式的计划、流程和结果报告。
- (3) 评审：开发组、测试组和相关人员联合进行的，采用讲解、提问并使用Checklist方式进行的查找错误的活动。一般有正式的计划、流程和结果报告。

动态测试（常义测试）：计算机必须真正运行被测试的程序，通过输入测试用例，对其运行情况（输入/输出的对应关系）进行检查和分析。

静态测试	动态测试
测试不运行的部分，只是检查和审阅（规范测试、软件模型测试、文档测试）	常义测试，运行和使用软件
通过文档评审、代码阅读测试软件	通过运行程序的测试软件
不用执行程序，主要用代码走查、技术评审、代码审查的方法进行测试	构造测试实例、执行程序、分析程序的输出结果进行测试

模拟用户的测试方法:

- (1) 基于对用户如何使用被测软件的了解来进行测试
- (2) 复杂软件产品有许多错误，但用户一般只能找出这些错误中的很少一部分
- (3) 要着重对哪些用户可能发现的错误进行测试和修改

评估方法:

代码覆盖

缺陷分析

图表

测试报告：记录软件测试过程、结果和结论的正式文档，用于向团队、管理层或客户清晰展示测试活动的执行情况和质量。测试阶段的最终交付物，帮助决策者判断软件是否达到发布标准。

核心内容:

模块	说明
概述	测试目标、范围、时间、参与人员、被测版本
测试环境	软硬件配置、网络环境、测试工具
测试执行总结	测试用例数、通过率、缺陷统计、测试覆盖率
缺陷分析	关键缺陷列表、修复状态、遗留风险
结论与建议	质量评估、是否达到发布标准、改进建议

